

ABRA — Technical Documentation

Version 7.0.0 • Last updated 2026-09-20

Written in ASD-STE100 Simplified Technical English. Sentences are short. The voice is active. One word has one meaning. The document follows the Diátaxis structure: Tutorial, How-to, Reference, Explanation.

This is the document that you follow to run this system. It tells you which command to give, which flag to give with it, and how to read what comes back. It does not argue for a design. The argument is in [the white paper](#). The record of how each figure moved is in [the running notes](#) and in [the changelog](#).

WHAT CHANGED IN 7.0.0. The MEDICHAM quarantine gate is OPEN. This edition folds in every running notes row from 6.0.1 to 6.83.0. It also removes the release-by-release evidence blocks that stood at the top of the 6.0.0 edition. Each figure in those blocks was measured on an engine that has since changed. Each one is superseded by section 0 below. **A superseded figure is deleted here. It is not captioned.**

0. Status — what this release publishes

0.1 The gate

Run this command. Read the first line of the output.

```
SHOWDOWN_PATH=/path/to/pokemon-showdown node engine/quarantine.js
```

It printed `GATE: OPEN – MEDICHAM passes both conditions; nothing is withheld on 2026-09-20. All ten gating clauses read PASS.`

Set `SHOWDOWN_PATH` **before you run the gate.** Two clauses load the official simulator: the board-leaf clause and the mechanics clause. Without the variable, both clauses print `CANNOT ANSWER` and the gate reads `CLOSED`. **CANNOT-ANSWER is not a pass. It is also not a defect.** It is a statement that the instrument did not run.

THE COMMAND EXITS 0 WHEN THE GATE IS CLOSED. The verdict is in the output. It is not in the exit code. Read the `GATE:` line. Then read each clause line. Do not decide from the exit status.

0.2 The ten clauses

clause, as the gate names it	what it reads
game differential	<code>data/engine-diff.json</code>
deliberate roster / items	<code>data/roster.items.json</code>
deliberate roster / abilities	<code>data/roster.abilities.json</code>
deliberate roster / moves	<code>data/roster.moves.json</code>
coverage — every used mechanic is measured by something	the differential and the roster
board leaves — nothing that can stand at a turn boundary goes uncompered	<code>tests/probe_uncompared_leaves.js</code>
whole-game differential / BOARD-MATERIAL, on every team lattice	the three lattice artifacts
whole-game differential / NARRATION, on every team lattice	the same three artifacts
mechanics — each one staged and compared against Showdown	<code>data/all-mechanics-fire.json</code>
no open, known engine defect	<code>docs/ROADMAP.md</code> , through <code>engine/register_reality.js</code>

The narration clause reports only while the board-material clause fails. It starts to gate when the board-material clause passes. Nobody sets this. `narrationClause` in `engine/quarantine.js` reads the verdict of the board-material clause on the same run.

0.3 The pins of this measurement

A measurement pins three things. It does not pin one. A release freezes the code. It does not freeze the store, the census or the artifacts. All three move.

what is pinned	value for this release
engine release	0d7b1d9db6d1
census pin file	data/verification/census-pin-b6df14beff8d.json
team store	data/team-pool-frozen
arm	middle
steering	empirical-click/v1

Read `engine_release` and `steering.input_read_from` in `data/game-differential.json` to confirm the first two. Read `steering.team_store_pinned_to` in the same file to confirm the third.

`--games` **is part of the definition of the sample. It is not a budget.** `engine/diff_swarm.js` picks teams by a stride. The step of the stride comes from `--games`. A larger value therefore plays DIFFERENT teams. It does not play the same teams and more of them. **Record `--games` in every report.** A run that does not record it cannot be repeated.

0.4 The readings

Whole games, board-material, on three team lattices. For each artifact, read `state.games` and `state.games_board_never_diverged`. Subtract the second from the first. The result is the board-material count. It is zero on each of the three.

`data/game-differential.json` records `games_requested` 1200 and `state.games` 961. `data/game-differential.g1350.json` records `games_requested` 1350 and `state.games` 1069. `data/game-differential.g1950.json` records `games_requested` 1950 and `state.games` 1497. Each of the three also reads zero for narration.

The held-out draw. The gate reads three lattices. A fourth draw is wider. It is taken on the same release, the same census pin and the same team store. It is not part of the gate. Its artifact is `data/verification/game-differential.g12000.json`. That path holds a directory, so the documentation gate cannot bind a figure to it. Read the artifact yourself. This is the readout:

```
$ node -e "const j=require('./data/verification/game-differential.g12000.json'); \
  console.log(j.games_requested, j.state.games, j.state.games_board_never_diverged, \
    j.state.games_before_void_exclusion, j.state.protocol_diverged_games)"
12000 7182 7182 7183 75
```

Board-material is the first value less the second. It is zero. One game is excluded as the instrument's own void game.

Board-material is zero on the wide sample and on the gate. That is new in this release. On each earlier release the wide draw found board partings that the three lattices could not see.

Damage. `data/engine-diff.json` records `requested` 6000, `compared` 6000, `agreed` 6000, `disagreed` 0 and `seed` 20260804. The gate prints the same zero at the midpoint of the damage roll and at every interior index.

The deliberate roster. `engine/legal_scope.js` decides which entities are in scope. Read `counts` in each stage artifact. `data/roster.items.json` reads 148 in scope, all of them FIRED-AND-BOARDS-MATCH. `data/roster.abilities.json` reads 196 of 200, with three rows accepted as ANNOUNCEMENT-ONLY on a receipt and one row DEFERRED-BY-OWNER. `data/roster.moves.json` reads 496 of 497, with one row DEFERRED-BY-OWNER. Each stage reads zero FIRED-AND-BOARDS-DIFFER, zero DID-NOT-FIRE, zero COULD-NOT-STAGE and zero CONTROL-NOT-QUIET.

The census. Read `data/mechanics-census.json`. It records `probed` 1002, `live` 1002, `armed` 1002, `missing` 0, `unarmed` 0, `hollow` 0, `threw` 0 and `run_ok` true. `node tests/test-mechanics.js` writes it.

The staged mechanics. Read `data/all-mechanics-fire.json`. It records `games_played` 4632, `games_threw` 0, `sheets_unassembled` 0 and `red_ok` true. Its `summary.moves` block reads `in_scope` 497 and `diverged` 0. Its `summary.abilities` block reads `in_scope` 200.

0.5 The limits of the verdict

Do not write "MEDICHAM is correct". Do not write "MEDICHAM is done". Write this instead: *every gating clause passes, including the blind-spot clauses*. Then give the limits below. **A clean verdict is a statement about a sample, a driver and an invented spread. It is not a statement about the ladder.**

- 1. Illusion is the one declared exclusion.** Each lattice drops the teams that carry it. Read `closet.teams_dropped` in `data/game-differential.json` : 43. Read the same field in `data/game-differential.g1350.json` : 63. Read it in `data/game-differential.g1950.json` : 71. The held-out draw drops more, in proportion to its size. A small minority of the sheets in the frozen pool carry a legal carrier of the ability, and fewer than half of those brought one. **Derive that share. Do not quote it from a document**, because no `data/*.json` artifact holds it. ROADMAP #160 holds the decision.
- 2. Closed team sheets and best-of-one play are out of scope.** No lattice samples them.
- 3. No differential body stands on a real spread.** An open team sheet does not show one. Read `declared_gaps.spreads_absent` in `data/game-differential.json` for the declaration. The nature is real where the sheet declares one.
- 4. Three roster rows are set aside by the owner.** The artifacts name them. They are not hidden.
- 5. A live reading of the last move is a known open class.** The official engine keeps the CALLING move. This engine keeps the called one. It is reported. It is not fixed.
- 6. The corner arms of the damage roll were not measured again for this release.**
- 7. Nothing downstream of MEDICHAM has been re-run.**

0.6 What an open gate does and does not do

An open gate makes an artifact below MEDICHAM **re-runnable**. It does not make that artifact **true**. `node engine/quarantine.js` lists each downstream artifact with the command that re-runs it. Each one was measured under an engine that has since changed.

Re-running is not optional. A withheld number does not become true when the gate opens. It becomes re-runnable. ROADMAP #57 holds the list.

1. Tutorial — run ABRA for the first time

Do these steps in order.

1. Get the code. Clone the repository `github.com/willhoop/abra`.
2. Get the sibling engine. Clone `github.com/willhoop/chomp` next to it. ABRA reads it at `../CHOMP`.

3. Get the official simulator. Clone and build a master checkout of `pokemon-showdown` next to them. Set `SHOWDOWN_PATH` to that directory. The Champions mod is not in the npm package.
4. Open the site. Open `web/index.html` in a web browser. The site needs no build step and no server.
5. Run one check. In a terminal, run `node engine/validate_damage.js`. It confirms the damage engine.
6. Read the state. Run `node engine/status.js`. Each figure in that output comes from an artifact. `NOT DERIVED` means that no artifact states it.
7. Read the open work. Run `node engine/open_work.js`. **Do not type a list of what is open. Print it.**
8. Read the gate. Run `SHOWDOWN_PATH=... node engine/quarantine.js`.

You now have the site, one validated model, the state of the project and the gate.

Run every heavy job through the wrapper. The wrapper starts the job at BELOWNORMAL priority.

```
tools\lownode.cmd engine\quarantine.js
```

Do not write `node engine\quarantine.js` for a heavy job. This machine runs many processes at once. A job at normal priority starves the window that you are typing into. The wrapper returns the exit code of the script. `tests/test-lownode.js` holds that rule.

2. How-to — common tasks

2.1 Measure the engine against the official simulator

Pin the three inputs. A release freezes the code. It does not freeze the store, the census or the artifacts.

```
node engine/game_differential.js --release <id> --census <pin file> \
  --team-store data/team-pool-frozen --arm middle --steering empirical \
  --end-state --games 1200 --turns 50
```

flag	what it pins
<code>--release <id></code>	the frozen engine sources
<code>--census <path></code>	the census that the run credits
<code>--team-store data/team-pool-frozen</code>	the pool of teams. The live store changes every hour
<code>--arm <id></code>	<code>middle</code> , <code>top-tie-first</code> or <code>bottom-tie-first</code>
<code>--games <n></code>	part of the definition of the sample. It is not a budget
<code>--turns <n></code>	the turn cap
<code>--end-state</code>	compare the state at the end of the game as well as at each turn boundary
<code>--write --out <path></code>	write the artifact to that path
<code>--dump-games</code>	write one record for each game. Use this to count a population

Record the flags with the result. Two runs at different `--games` values are two different questions under one artifact name.

Measure the three lattices. The gate reads three samples. Run the differential three times on one release with the same pins. Change only `--games` and the output path. `latticeRerun` in `engine/quarantine.js` prints the command.

```
node engine/game_differential.js --steering empirical --release <id> --arm middle --end-state \
  --census data/mechanics-census.json --team-store data/team-pool-frozen \
  --games 1200 --write --out data/game-differential.json
# then --games 1350 --out data/game-differential.g1350.json
# then --games 1950 --out data/game-differential.g1950.json
```

Take the held-out draw. It is wider than the gate. It uses the same pins. It is not part of the gate.

```
node engine/game_differential.js --steering empirical --release <id> --arm middle --end-state \
  --census <pin file> --team-store data/team-pool-frozen \
  --games 12000 --dump-games --write --out data/verification/game-differential.g12000.json
```

Prove that two samples are the same before you compare them.

```
node engine/arms_comparable.js <before.json> <after.json>
```

It refuses a pair that is not comparable. Compare the number of games, the list of first divergences and the coverage block. Do not assume that two runs sampled the same population.

Write the prediction before the run. Say which games each fix must remove, and from which lattice. Compare after the run. A game that joins a lattice is a new finding. A predicted game that stays is a fix that did not work.

2.2 Read the differential without the four traps

Read the clause, not the field whose name looks right. The board-material bar is `state.games` less `state.games_board_never_diverged`. `by_cause_totals.games_board_material` is the by-cause attribution. It is a different number. Check both operands of every subtraction. Name the clause that you read.

The capped lists are not the population. `state.first_board_divergences` holds the first forty rows. `state.first_divergences` holds the first sixty. A truncated list is evidence of what EXISTS. It is never evidence of how much. Bucket the `--dump-games` output to count a population.

The by-cause list is keyed on the first PROTOCOL divergence. The bar reads the first BOARD one. Do not aim a batch of fixes from the wrong list.

Do not read an artifact while another process writes it. A torn read is not an error. It is a well-formed answer that is false. Check the modification time against the clock first. Prefer `git show HEAD:<file>` for a stable read.

2.3 Read the gate and its clauses

```
SHOWDOWN_PATH=... node engine/quarantine.js           # the gate, the failing clauses, the withheld set
SHOWDOWN_PATH=... node engine/quarantine.js --graph   # why each artifact is in or out
SHOWDOWN_PATH=... node engine/quarantine.js --check   # fails if a withheld figure is printed
SHOWDOWN_PATH=... node engine/quarantine.js --whole-game # the board-material clause alone
SHOWDOWN_PATH=... node engine/quarantine.js --narration # the narration clause alone
SHOWDOWN_PATH=... node engine/quarantine.js --order-probe # the move-against-move turn-order floor
node engine/quarantine.js --selftest                  # drive each branch on synthetic input
```

The two whole-game commands answer two questions. Neither is "the" divergence rate. `--whole-game` counts games whose boards part. `--narration` counts games whose protocol parts while the board does not. Neither total can be derived from the other. Quote one with its name.

A lattice that is missing, stale, not pinned, labelled wrongly, or equal to another lattice gives CANNOT-ANSWER. `engine/game_differential.js` writes `games_requested` into the artifact, so the sample is recorded and is not inferred. The eight fields that must agree across the three artifacts are `LATTICE_SAME` in `engine/quarantine.js`.

A declared divergence is subtracted by name. The declarations are `DECLARED_DIVERGENCE` in `engine/quarantine.js`. Each one carries a reason.

2.4 Regenerate the census and the staged mechanics

```
SHOWDOWN_PATH=... node tests/test-mechanics.js          # writes data/mechanics-census.json
SHOWDOWN_PATH=... node engine/all_mechanics_fire.js --release <id> --kind all --write
```

Pin the census before you use it in a measurement. Copy the live census to `data/verification/census-pin-<digest>.json`. Then give that path to `--census`. A census that gains a row changes which scenarios play, so a count taken on either side of a regeneration is not a before-and-after.

`all_mechanics_fire.js` **without** `--release` **cuts a release over the live tree and says so.** That is a smoke run. It is not a measurement.

The census counts the probes that are registered. Register a probe above the point where the file takes its snapshot. A probe registered below that point is not counted, and a missing row is then invisible.

2.5 Stage a roster row

```
SHOWDOWN_PATH=... node tests/roster.js --stage items    --reds --write
SHOWDOWN_PATH=... node tests/roster.js --stage abilities --reds --write
SHOWDOWN_PATH=... node tests/roster.js --stage moves    --reds --write
```

The stage builds two arms for each row: the subject arm and the control arm. The control arm differs from the subject arm in one mechanic and in nothing else.

- **If the control arm can act on the board, the row is CONTROL-NOT-QUIET. That is not a green.**
- The quiet set is decided for each arm by `quietAsControl(id, arm)`. A blocker of critical hits is not quiet on the arm where every critical hit lands.
- The idle click is a move that the fixture body can learn. Each run prints `THE CONTROL CLICK` for each stage and an alarm for a sleeping body.
- A row that the owner set aside reads DEFERRED-BY-OWNER. Do not stage it. Do not count it as a gap.
- A row whose whole effect is a printed line is accepted only on a receipt. The receipt names the knob that makes the row red and the probe that proves it. A row whose knob cannot make it red is refused.
- `--reds` plants a defect for each shape rule and reads whether the stage catches it. **A red that reads NOT CAUGHT means that every green of that rule proves nothing.** Correct the plant or the rule before you publish a count.
- **The anchor audit runs on every roster run.** A red demonstration plants a break by an exact string replace and must match exactly one time. A dead anchor is an ordinary failing row. It reaches the exit code.

2.6 Ask whether a mechanic is in the regulation

`node engine/legal_scope.js` prints the derivation. In code:

```
const S = require('./legal_scope.js').derive(); // memoised for each process
S.inScope('ability', id);                      // true or false
S.verdict('move', id);                        // { inScope, code, why, ... }
S.why('item', id);                            // the reason it is out, or null
```

This file is the one implementation of scope. `engine/coverage.js`, `engine/stage_planner.js`, `engine/all_mechanics_fire.js`, `tests/roster.js` and `engine/tag_dex.js` all ask it. **Do not write a second scope rule.** Two rules for one fact disagree in the end, and the disagreement is not visible. The codes are in the Reference, under *Scope codes*.

2.7 Plan a legal fixture for a mechanic

```
node engine/stage_planner.js                # plan each mechanic, print the coverage
node engine/stage_planner.js --only ability:<id> --show
node engine/stage_planner.js --json <path>   # write the plan to a scratch path
```

The planner plays no games. It reads what a mechanic needs from the handlers of the official engine and from the tag parameters. Then it builds a legal board that supplies that need, and a control that differs in one leaf. It asks the official `TeamValidator` about each team that it emits.

A COULD-NOT-STAGE result is a statement about the fixture. It is not a statement about the mechanic. Construct the fixture. Do not look for one.

Run `node tests/test-stage-planner.js` after you change the planner. `STAGE_PLANNER_BREAK=<mode>` runs a planner that is broken on purpose, to show that the test can go red.

2.8 Check that a fixture body is legal

`engine/fixture_legality.js` asks the official `TeamValidator` about each body that a test builds. It also checks status codes and genders against the species. It reports. It does not fail a run. A directed game hashes the same with the check on and with it off.

Ask the validator. Do not walk the learnset. A raw learnset walk accepts a body that the format refuses, and a probe on an illegal fixture is red for the wrong reason.

2.9 Work with a frozen release

```
node engine/engine_release.js cut "why this release exists"
node engine/engine_release.js list                        # each release, and how many files moved since
node engine/engine_release.js verify <id>                # has the copy itself changed
node engine/engine_release.js compat <file> [symbol ...] # can this release still serve this caller
node engine/engine_release.js rerender <id>               # draw release.json again from the cut log
```

Read the frozen set from `SOURCES` in `engine/engine_release.js`. **Do not copy that list into a document.** It has grown four times.

```
node -e "console.log(require('./engine/engine_release.js').SOURCES.join('\n'))"
```

- **A cut refuses if the official checkout is not at** `PINNED_COMMIT` . `--allow-authority-drift` records the drift in the cut event. Use it deliberately.
- **A second cut over an identical tree appends. It does not overwrite.** Cuts are events in `data/releases/<id>/cuts.jsonl` . The top-level `cut` and `why` mean the first freeze. They are never rewritten.
- **A release freezes the engine and not the reader.** Each symbol that a caller adds to its list of needs strands every release cut before it. The release still verifies and stops being openable. Ask `compat` before a run finds the answer deep inside itself. **A stranded artifact is a figure to withhold and to measure again. It is not a thing to repair.**
- **A published figure may cite only a release that is in the repository.** `engine/provenance.js` holds this as RULE 5. The check asks the manifest of the release, not the directory, so a release that is tracked in part also fails.

2.10 Read which configuration produced a result

```
node engine/run_stamp.js --show data/<artifact>.json
```

Every gate artifact has a `<name>.meta.json` file beside it. Read three fields before you quote a result. `reconstructed: true` means that the stamp was inferred from a commit and not observed; read `confidence` beside it. `git.dirty: true` means that the commit does not describe what ran; use `source_digests` instead. `source_digests` holds hashes of working-copy bytes and `git.blobs` holds git object names. **Do not compare the two.** On Windows they differ, because git changes the line endings.

Write a stamp from a new measurement with `require('./run_stamp.js').writeStamp({...})` , at the point where the run writes its numbers. **Do not write a second sidecar format.**

2.11 Check whether an artifact is too old to trust

```
node engine/provenance.js
```

The report gives a drift percentage and, beside it, `ci_gain` and `max_shift` . **Use** `max_shift` . It states how far the missing games can move the result. A percentage of a store that grows every hour states only the AGE of the artifact.

A receipt is written from the path that the run opened. It is never written from the canonical path for that kind of file. A receipt that names the wrong store passes this check for the wrong reason, because the check judges each key against the file that the key names.

2.12 Commit a change

Arm the hook one time for each clone: `git config core.hooksPath .githooks` .

The hook judges the commit. It does not judge the working tree. Each gate reads a file as the commit will hold it: the staged version, or the version in `HEAD` when the file is not staged. `useIndex()` in `engine/docs_scan.js` is the one reader. It obeys `GIT_INDEX_FILE` .

gate	what the hook runs	when
silent catch	<code>node tests/test-no-silent-failure.js --only <staged files></code>	a staged <code>.js</code> file at the top level of <code>engine/</code> , <code>build/</code> or <code>tests/</code>
generated bundle	<code>node engine/artifact_audit.js --staged</code>	each commit
running notes	<code>node engine/docs_scan.js --note-check <staged files></code>	a commit that touches <code>docs/</code> , <code>engine/</code> , <code>tests/</code> , <code>web/</code> , <code>CHANGELOG.md</code> , <code>README.md</code> or <code>CLAUDE.md</code>

gate	what the hook runs	when
living docs	<code>node tests/test-docs-current.js --staged</code>	the same
roadmap register	<code>node tests/test-roadmap-register.js --staged</code>	the same
re-runnability	<code>node tests/test-artifact-rerunnable.js --staged</code>	the same

The hook does not run during a rebase. A run by hand without `--staged` reads the working tree. **Do not use `--no-verify`**. If a gate is wrong, correct the gate and say so.

2.13 Add a figure to a living document

The check on an untraceable figure is a ratchet. A figure is traced only by a trace that is bound to it. There are three forms.

1. An artifact that the paragraph of the figure cites, and that holds the value. If the citation names a field, the value must be in that field.
2. The CHANGELOG entry that the block of the figure names.
3. A commit-pinned citation: `<commit>:data/<file>.json`, or `<commit>:data/<file>.json:<field>` for one field. The check reads those bytes from git, and not from the file of today.

A hit somewhere else in `data/` is not a trace. A hit somewhere else in the CHANGELOG is not a trace.

RULE. Put the citation in the same sentence as the figure. Do not write a figure into the CHANGELOG to give it a trace.

RULE. A figure that a newer measurement supersedes comes OUT of the document. Delete it. Do not write a caption beside it. A caption is not a quarantine, and this project has twice proved that a printed warning beside a number is skimmed past.

2.14 See what the next major release owes

```
node engine/docs_scan.js --owed      # the backlog, and how near it is to the cap
node engine/open_work.js             # the same block, beside the open register rows
```

The cap is `OWED_CAP` in `engine/docs_scan.js`. Above the cap, the build fails. Clear the backlog with a documentation pass **at any version. Do not make a release `X.0.0` to empty a backlog**. A major release is declared by a change of basis and by nothing else.

2.15 Run the stores

```
PAGES=6 CONC=20 node engine/durable-ingest.js data/games.ladder.jsonl
PAGES=6 CONC=20 FORMATS=gen9championsvgc2026regmbbo3 node engine/durable-ingest.js data/games.bo3.jsonl
MODE=backfill node engine/durable-ingest.js data/games.ladder.jsonl
node engine/analyze.js data/games.ladder.jsonl          # writes data/meta-usage.json
python3 engine/dedupe_store.py data/games.bo3.jsonl --write
```

- The pull adds only new games. It never duplicates a game. It never re-fetches a stored game.
- `MODE=reparsing` REFUSES to run while any stored game has no raw log. Run `MODE=backfill` first.
- **The best-of-three store is the open-sheet ladder.** Its ruleset carries `Force Open Team Sheets`, so every game publishes all six sets of both sides. **Keep it in its own store. Never pool it with the ladder store.** The two are different information regimes.

- **The stores are sharded and the shards are tracked.** State the growth budget in any change that touches them. Check what is TRACKED with `git ls-files`, not what is on disk. GitHub rejects any single file above 100 MB. That is a push failure, not a warning.
- **Do not repair a store row by hand.** `data/quality-filter.json` holds an exclusion rule with a reason code. `engine/quality.js` and `engine/quality.py` apply it as the last stage of the funnel. **It is a declaration. It is not a detector.** `python engine/sanity_check.py` fails on a bad row that is not declared, and on a declared row that is correct now.

2.16 Play and fit

```
SHOWDOWN_PATH=... node engine/champions_sim.js           # the official engine
SHOWDOWN_PATH=... node engine/mew.js --n 1000 --policy score # self-play
SHOWDOWN_PATH=... node engine/mew_farm.js --n 200000 --procs 12 --conc 1
SHOWDOWN_PATH=... node engine/validate_selfplay.js
SHOWDOWN_PATH=... node --max-old-space-size=4096 engine/fit_policy.js
SHOWDOWN_PATH=... node --max-old-space-size=4096 engine/fit_joint.js
```

- Self-play output goes to `data/games.selfplay.jsonl`. **Never pool it with a human store.**
- Two files are written for a farm run, and both are needed: the game records, and `data/games.selfplay.raw-logs.jsonl`, which holds the full protocol logs. The value models replay the protocol log. A corpus written without the sidecar cannot be read by the models that it exists to train.
- `--conc` must stay at 1. The simulator is synchronous and CPU-bound. Process count is the unit.
- Pass the same `--seed` to both arms of a comparison. The two corpora then play identical teams.
- **The fitting environment and the playing environment must match.** A model fitted with the team sheet visible must not play without it.
- `fit_policy.js` and `fit_joint.js` need `node --max-old-space-size=4096`.
- **Every figure from a model downstream of MEDICHAM is re-runnable and is not current.** Do not quote one from a document. Re-run it first.

2.17 Three shell traps on this machine

These three cost real time on 2026-09-20. Each one made a green run look red.

1. A `cmd /c` run that is redirected to `/dev/null` reports exit 1 on a GREEN run. `cmd.exe` cannot write to that path. **Redirect to a real file.** Then read the file.
2. A double-quoted Windows path that is built with `printf` can turn a backslash escape into a carriage return. `printf "tests\roster.js"` asks for `tests<CR>oster.js`, and node reports that the script is missing. A double-quoted string can also leave a shell variable unexpanded. **Use single quotes, so that each backslash stays literal.**
3. **Before you believe any red, read the OUTPUT of the run. Do not read its exit code.** Eight false reds were produced in one day by the two traps above. Nothing was changed on the strength of any of them, because each was read from the output.

And one more, from the gate section above. `node engine/quarantine.js` exits 0 when the gate is CLOSED. The verdict is a line of text.

2.18 Other tasks

Compare the engine with the official simulator on damage alone. `SHOWDOWN_PATH=... node tests/test-engine-diff.js --n 6000 --seed 20260804`. The sampler is seeded. Two runs with the same seed give the same result. **Always record the seed with the count.**

Check the engine against the project standards. `node engine/conformance.js` . Standard S12 is that nothing is written in code which belongs in a configuration file. Standard S13 is that no state is kept by hand. **A test derives the format id from** `engine/champions_sim.js` . **Do not write the format id in a test.** After the regulation changes, a format id that is written by hand tests the earlier game.

Check the register against its instruments. `node engine/register_reality.js` , and `node engine/register_reality.js --selftest` after you change it. A CANNOT-ANSWER result is not a disagreement. **A closed row must name an instrument that a person can run.**

Know which tags a source file reads.

```
const { sourceConsumers, lookupCalls } = require('./tag_lookups.js');
sourceConsumers(src);    // the tag names given as a literal argument of a TAGS lookup
lookupCalls(src);        // { calls: [...], unclosed: [...] }
```

A `)` inside a string, a template, a regular expression or a comment does not close a call. A call that does not close is reported in `unclosed` . It is never dropped without a report.

Find where a fact lives. `node engine/where.js <thing>` . It derives the answer at run time from `SOURCES` , the tests and `data/tags.json` . **A written map of this repository is wrong within a day.**

Validate the damage engine. `node engine/validate_damage.js` . It fails if any scenario is more than 5% from the Smogon damage calculator.

Show the project state on a web page. `node web/build-status.js` , then open `web/status.html` . The page reads a script-tag global. **Do not change it to** `fetch()` . A `fetch()` of a local file fails under `file://` and shows no error to the reader.

Check that every room parses. `node tests/test-web-parses.js` . A page can hold correct text and still fail to run.

Refresh the site data. `python3 engine/refresh-site-data.py` . It writes `data/live.js` , `data/archetypes.json` and `data/kad-replays.js` .

Edit the site, then mirror it. After you change `web/index.html` , copy it to `app/index.html` . Run `node tests/test-site-sync.js` .

Stamp the generated blocks. `node engine/status.js --write` . **Never hand-edit inside a** `<!-- GENERATED -->` **block.** **Never write a handoff document.** State is printed. It is not typed.

3. Reference

3.1 Stored game record

`data/games.ladder.jsonl` and `data/games.bo3.jsonl` hold one JSON object for each line. The stores are sharded. `engine/durable-ingest.js` is the source of truth for the schema; it exports `extract()` .

field	meaning
<code>id</code> , <code>date</code>	replay id and upload time
<code>p1</code> , <code>p2</code>	<code>{name, rating, bot}</code> for each player
<code>six.p1/p2</code>	the six revealed at team preview
<code>brought.p1/p2</code>	the four actually brought
<code>lead.p1/p2</code>	the two led
<code>sets</code>	for each species, the revealed moves, item and ability

field	meaning
turns	per-turn events: move, damage, faint, status, field
winner	the name of the winner

Store raw. Analyse on top. Every filter runs over the store at read time. A change to how the games are segmented is a re-computation. It is never a re-download.

3.2 The artifacts that the gate reads

artifact	written by	contents
data/game-differential.json	engine/game_differential.js --games 1200	the first team lattice
data/game-differential.g1350.json	the same, --games 1350	the second team lattice
data/game-differential.g1950.json	the same, --games 1950	the third team lattice
data/verification/game-differential.g12000.json	the same, --games 12000	the held-out draw. Not part of the gate
data/engine-diff.json	tests/test-engine-diff.js	the damage differential
data/roster.items.json	tests/roster.js --stage items	one staged scenario for each item in scope
data/roster.abilities.json	tests/roster.js --stage abilities	one staged scenario for each ability in scope
data/roster.moves.json	tests/roster.js --stage moves	one staged scenario for each move in scope
data/mechanics-census.json	tests/test-mechanics.js	which probes are registered, live and armed
data/all-mechanics-fire.json	engine/all_mechanics_fire.js	each mechanic staged and compared
data/protocol-events.json	engine/derive_protocol_events.js	every event Showdown can emit, and which of them this engine emits
data/tags.json	engine/tag_dex.js	the derived tag set that the engine reads

Other artifacts that a reader asks for often:

artifact	written by	contents
data/meta-usage.json	engine/analyze.js	the usage model that CHOMP reads
data/damage-validation.json	engine/validate_damage.js	damage error against the Smogon damage calculator
data/move-priors.json	engine/move_priors.js	P(move given species), from recorded human clicks
data/bring-priors.json	engine/bring_priors.js	P(brought given on team), and P(lead given brought)
data/quality-filter.json	by hand, as a declaration	rows excluded from the store, each with a reason code
data/test-waivers.json	by hand, on the owner's word	each waived red, with the words and the date

Every figure from a model below MEDICHAM is re-runnable and is not current. Run `node engine/quarantine.js` for the list and for the command that re-runs each one.

3.3 Protocol trace (`engine/medicham2-browser.js`)

The simulator can emit a Showdown-shaped protocol stream. It is **off by default**. Nothing in the battle loop pays for it unless a caller asks.

```

const trace = [];
const S = M.battleInit(teamA, teamB, { trace });
M.battleTurn(S, rng);
// trace is an array of protocol lines:
//   |move|p1a: incineroar|fakeout|p2a: garchomp
//   |-damage|p2a: garchomp|154/175
//   |cant|p2a: garchomp|flinch

```

export	does
M.TRACE_EVENTS	the event names that this engine claims it can emit
M.traceCounts(lines)	counts by event name, PARSED from the lines rather than kept beside them
M.traceCanon(line)	the one normaliser. It lowercases and strips whitespace for each field

data/protocol-events.json records emittedCount 46 and notEmittedCount 48. **Read the artifact. Do not read a number out of this paragraph.**

Identifiers are ids. They are not display names. This engine writes p1a: incineroar and fakeout where Showdown writes the display forms. This engine holds no display-name table. A translation layer can itself be wrong. traceCanon() is applied to **both** streams by any comparison driver, so the two agree by canonicalisation and not by translation.

Do not add an event without regenerating the artifact. node engine/derive_protocol_events.js --write fails if a name in TRACE_EVENTS is one that Showdown never emits. It also fails if a Showdown event is neither emitted nor given a reason. tests/test-protocol-trace.js fails if a claimed event never fires in a real game.

3.4 Scope codes (engine/legal_scope.js)

verdict(kind, id) gives one of these codes. Each part is read from the format. No part is written by hand.

code	kind	meaning
CARRIED	ability	a legal species has the ability, and the official TeamValidator accepts that species with it
LEARNED	move	a legal species can learn it, by the move pool that the validator reasons from
INJECTED	move	the simulator gives it to a body that has no learnset for it
CONFERRED	ability	no accepted carrier, but a source in scope writes the ability onto a body
HELD	item	legal. A mega stone counts only while one of its mega formes is legal. An item with a named user counts only while one of those users is legal
NO-LEGAL-CARRIER	any	nothing legal carries, learns or holds it, and nothing in scope confers or injects it
VALIDATOR-REFUSED	any	legal species carry it, and the validator refuses each one
NO-LEGAL-READER	ability, item	each handler writes only a value that nothing in the regulation reads

3.5 Exit codes (engine/exit_codes.js)

exit code	what it means
0	a green verdict, unless a declaration contradicts it
1	a red verdict. Node also exits 1 on an error that is not caught
any other code	not a verdict, unless the instrument declares one at the start of a line: ABRA-EXIT <code> CANNOT-ANSWER or ABRA-EXIT <code> VERDICT-RED
no code	not started. The process did not run, or a signal stopped it

```
const EXIT = require('./exit_codes.js');
EXIT.classifyExit(status, output);    // { green, kind, declared, why }
EXIT.runnerOutcome(status, output);  // the same, plus PASS | FAIL | SKIP
EXIT.declaration(code, kind);        // the line an instrument prints as it stops
```

`runnerOutcome` adds a policy only where the classifier says that there is no verdict. A declared `CANNOT-ANSWER` becomes `SKIP`. An undeclared exit code 2 becomes `SKIP`. Each other unclear result becomes `FAIL`. **A crash is not a skip.** `node engine/exit_codes.js --selftest` drives each branch.

An exit code is not a verdict on its own. The gate section and the shell-trap section give four cases where the exit code and the output disagree.

3.6 Quarantine classifier routes (`engine/quarantine.js`)

The classifier holds or releases an artifact by a derived rule. It does not read a list of names.

route	what it decides
<code>sideBySideInstruments</code>	a generator that needs MEDICHAM and the official engine together, and that reaches no <code>board.js</code> , is an instrument. An instrument is not withheld
<code>foreignSource</code>	a Python generator is read for the dumps that it names, which MEDICHAM played
<code>describedBySelf</code>	an artifact with no writer that can be found is held by what it says about itself

`engine/provenance.js` does not count a write into a temporary directory as a write into `data/`.

3.7 Engine knobs — the convention

Every engine correction ships with a named knob that restores the earlier behaviour, and with a probe that is red under that knob. The knob is an environment variable. Its name starts with `MEDI_`. A knob with no effect on any input is the finding, not a pass: it means that the code is not wired.

There is no table of knobs in this document, and that is deliberate. The set grows on most days. A typed list of it goes stale in the same way as the fourteen handoff documents and the ban list of four. Derive it:

```
# every knob the simulator reads
node -e "const s=require('fs').readFileSync('engine/medicham2-browser.js','utf8');\
console.log([...new Set([...s.matchAll(/MEDI_[A-Z0-9_]+/g)].map(m=>m[0]))].sort().join('\n'))"
```

`DELIBERATE_BREAK` in `tests/test-mechanics.js` lists the restore functions that a red demonstration may set. **A run that sets one of them refuses to write the census.** That is what stops a red demonstration from publishing itself as state.

To reproduce a correction, set its knob and run its probe. The probe must read clean without the knob and red with it.

```
SHOWDOWN_PATH=... node tests/probe_<name>.js
SHOWDOWN_PATH=... MEDI_<KNOB>=1 node tests/probe_<name>.js
```

3.8 Instruments

file	the question that it answers
<code>engine/quarantine.js</code>	is the gate open, and what is withheld
<code>engine/status.js</code>	what is the state of the project now
<code>engine/open_work.js</code>	which register rows are open, and which defects an instrument measures
<code>engine/where.js</code>	which file owns this fact, and which instrument decides this question

file	the question that it answers
engine/provenance.js	is this artifact too old, or resting on a release that is not tracked
engine/game_differential.js	do the two engines hold the same board through a whole game
engine/diff_swarm.js	which teams does a given --games value play
engine/arms_comparable.js	did these two runs sample the same population
engine/engine_release.js	freeze, verify and open a set of engine sources
engine/legal_scope.js	is this mechanic in the regulation, and if not, why not
engine/stage_planner.js	what legal board makes this mechanic fire, with a control
engine/all_mechanics_fire.js	does each staged mechanic fire and agree
engine/tag_dex.js	which tags does the regulation produce
engine/tag_lookups.js	which tag names does this source file look up
engine/exit_codes.js	what does this exit code mean
engine/register_reality.js	does each register row agree with the instrument that it names
engine/fixture_legality.js	is each fixture body legal, with a legal status and gender
engine/conformance.js	does the code obey the project standards
engine/coverage.js	which mechanics are measured by something
engine/docs_scan.js	what do the documents owe, and which figure is bound to nothing
engine/artifact_audit.js	are the values of a source actually in the file that it generates
engine/mod_audit.js	did Champions change this ability, move or item
engine/major_readiness.js	which withheld artifacts would become quotable on a re-run
tests/roster.js	does each entity in scope fire, against a quiet control
tests/test-mechanics.js	which probes are registered, live and armed
tests/test-stage-planner.js	is each fixture legal, masked for no reason, and inert for one reason
tests/probe_uncompared_leaves.js	can any leaf stand at a turn boundary uncompared
tests/probe_heldout_board_partings.js	do the partings of the held-out draw stay closed
tests/probe_reopen_partings.js	do the partings that the planner found stay closed
tests/test-engine-consistency.js	do the two engines agree about the FACTS
tests/test-engine-release.js	does an open release still serve its own bytes after a live edit
tests/test-artifact-rerunnable.js	can each published artifact still be re-run
tests/test-docs-current.js	is each figure in a living document generated, cited or deleted
tests/test-wiring.js	did each capability actually run in a real game
tests/test-lownode.js	does the priority wrapper return the exit code of the script

A capability that cannot prove that it ran is assumed broken. Every capability emits a counter. The run prints the counter. A zero is called out. tests/test-wiring.js plays real games and FAILS when a counter is zero.

3.9 Continuous collection

A GitHub Action (.github/workflows/ingest.yml) runs the pull every hour and commits the store. A second workflow collects the next regulation. A tests workflow runs the suite and the damage validation on every push and pull request.

Only one agent may run git from a keyboard. The bot commits from the workflow, into data/ and docs/ORIENTATION.md and nowhere else. The two are race-safe by a shared concurrency group.

4. Explanation

These sections explain a decision. They do not give instructions.

4.1 Why a measurement pins three things

A release freezes a declared set of engine sources. It is a copy, not a checksum. A measurement that reads the copy is safe from another division rewriting the simulator while it runs.

That is necessary. It is not sufficient. A release does not freeze three things that move.

- **The store.** The ingest appends to the ladder store and to the best-of-three store every hour.
`engine/game-differential.js` draws its team pool from those files. Ask for the same number of games one hour apart, and you get two different samples with no warning.
- **The census.** `engine/all-mechanics-fire.js` is steered by `data/mechanics-census.json`. A census that gains a row changes which scenarios play.
- **The artifacts.** Any run rewrites `data/game-differential.json` and `data/all-mechanics-fire.json`.

So a measurement pins `--release`, a census pin file, and `--team-store data/team-pool-frozen`. Then it PROVES that two samples are the same, rather than assuming it.

A measurement is a photograph. Nothing in the frame may move, including files that the measuring agent never opens. On 2026-08-04 three divisions ran at the same time with separate files. The weights of the model under test changed between the two halves of one measurement. The measurement completed. No check failed. Thousands of games were lost.

4.2 Why `--games` is part of the sample

`engine/diff_swarm.js` picks teams by a stride whose step is computed from `--games`. A larger run therefore plays DIFFERENT teams. It does not play the same teams and more of them.

The consequence is exact and it was measured. On one release, one census pin and one team store, the three lattices held different populations of teams and gave different counts of board partings. **A zero that holds at one sample size is a fact about that sample.** The gate therefore requires zero at three values of `--games`, chosen by walking the swarm builder so that the samples share few teams. A fourth value was measured and rejected, because it re-picks most of the teams of the first.

This is also why the held-out draw exists. It is wider than the gate, on the same pins. On each release before this one it found board partings that the three lattices could not see.

4.3 Why board-material is the bar, and narration is a second gate

The owner decided this on 2026-08-22: **board-material now, narration as its own separate gate afterwards**. The distinction is one that the engine already enforces elsewhere: *commentary may differ; boards may not*.

This is not a relaxation. What it removes from the critical path was measured, not assumed. Some divergences write no board leaf at all. That is a fact that an instrument reports. *"That is just a message"* is a guess, and a guess is not evidence.

The narration gate is a gate, not a backlog. It has its own clause and its own count, so that *"we will do narration later"* cannot become an unbounded deferral. The clause starts to gate the moment the board clause reads zero. Nobody sets that. It is computed.

4.4 Why the pinned pool and the lab answer different questions

The pinned pool (`data/team-pool-frozen`) is a frozen snapshot of real ladder games. It holds what people actually brought, so it is weighted by usage by construction. It answers *does this matter*.

The roster, the census and the staged mechanics are a **lab**. They stage one deliberate scenario for each entity, whatever its usage. They answer *is this correct*.

A mechanic that is missing from the pool is not a hole in the pool. It is a fact about the metagame. Four correct engine fixes once moved the whole-game rate not at all, because the pool holds no carrier of the mechanic that they fixed. The lab saw every one of them. Nothing was wrong except which scoreboard was quoted.

So: before you start on a mechanic, say which scoreboard it should move, and check that one. A common mechanic should move both. A rare mechanic should move the lab and leave the pool still. Say that before the run, not after it.

4.5 Why a capability that cannot prove it ran is assumed broken

Every serious defect found on 2026-07-28 had one shape: **a capability was absent, and everything reported success**. No exception, no failed test, no discarded game.

A head-to-head, an exploitability search and a prediction score all compare two bots that SHARE the blind spot. A missing capability therefore cancels out exactly. Only a person looking at the screen found them.

Two rules follow.

- **Every capability emits a counter. The run prints it. A zero is called out.**
- **Non-zero is not always a strong enough bar.** Where a domain rule exists, it becomes a RATE floor.

4.6 Why facts are global and features are per-model

The models answer differently-shaped questions, so they must NOT share a feature vector. A model that scores an ACTION and a model that scores a POSITION are not the same question, and feeding action-features to a position model is a category error.

What they must NEVER each own is a **fact** about the game: how much damage this does, who moves first, whether that ability refuses this move, what the sheet declared. One implementation, and everybody calls it. Two files that both decide what a Choice Scarf does to Speed will disagree in the end, and the disagreement will be invisible, because both keep working.

A legitimate exception, so that the rule is not misread: `engine/board.js` computes EXPECTED speed across unknown spreads, and `engine/medicham2-browser.js` computes EXACT speed for a built body. Those are two questions. The MULTIPLIERS underneath them are the fact, and they must be one function that both call. `tests/test-engine-consistency.js` asserts that the facts agree across engines. It deliberately says nothing about whether the features match.

4.7 Why a derived artifact is not a fact until something compares it to its source

A fact reaching every MODEL is one rule. A fact reaching the ARTIFACT that carries it is a second rule, and it is a separate hole. A generated file needs a check that the values of its source are actually in it.

`engine/artifact_audit.js` is that check, and it is registered as a gate.

Three things that the check must do, each learned by getting it wrong first.

1. **Judge a builder only on the rows that it actually WRITES.** Averaging over rows that it skips will clear the builder that is broken.
2. **Ask whether the ARTIFACT has two keys that normalise alike.** Do not ask whether the two files spell keys the same way. Spellings differ for good reasons. Duplicates never do.
3. **Treat "newer than its source" as no evidence at all.**

4.8 Why additivity is the recurring failure

A sum of independent terms cannot express a conjunction. If a score is $w_1*a + w_2*b$, there is no setting of the two weights that makes "a AND b together" worth more than the parts. That is not *scored badly*. It is **not representable**.

The same defect has appeared at three levels of this project: within one Pokemon, across a pair of Pokemon, and across a team of six. The literature names the boundary. Cooperative multi-agent value factorisation (QMIX, Rashid et al. 2018) constrains the joint value to be monotonic in the utility of each agent, and QPLEX and Weighted QMIX exist because that constraint cannot represent non-monotonic coordination. This case is the non-monotonic one.

The expensive machinery does not apply here. That literature avoids enumeration when agents are many. With two Pokemon the coordination graph is a single edge, so variable elimination degenerates to enumerating the joint actions, and message passing is unnecessary.

And the fix is not "add the interaction term". It is "fit it for the right thing." A pair block that is fitted to RESEMBLE human pairs is not fitted to WIN.

Practical rule. Before you add a feature, ask whether the thing you want to say is a conjunction. If it is, no weight on an individual term will ever say it.

4.9 What a neural network is here, and why it is not automatically an upgrade

Every value model in ABRA answers one question: given the board now, what is the probability that I win? A logistic regression answers it by multiplying each feature by a weight, adding them, and squashing the sum to the range zero to one.

$p = \text{sigmoid}(w_0 + w_1\text{alive_diff} + w_2\text{hp_diff} + \dots)$

That can draw only a straight dividing line through feature space. It cannot express "*being one Pokemon ahead matters enormously on turn 3 and barely on turn 25*", because that is an INTERACTION between two features.

A neural network is the same idea with a middle step.

$h = \text{relu}(W_1 @ x + b_1)$ # learned intermediate quantities $p = \text{sigmoid}(W_2 @ h + b_2)$

Each hidden unit can become a detector for a conjunction, and the output layer weighs the detectors. Universal approximation (Cybenko 1989; Hornik 1991) says that one hidden layer of sufficient width can represent any continuous function on a bounded domain. The network is therefore strictly **more expressive** than the linear model.

Strictly more expressive is a claim about representation. It is not a claim about learning. Extra capacity spent on features that hold no interactions buys nothing and costs variance. If the features carry no more signal, a network fitted to them lands in the same place.

This is the lesson that the game-playing literature learned repeatedly. **TD-Gammon** (Tesauro 1994) needed hand-designed board features. **AlphaGo Zero and AlphaZero** (Silver et al. 2017, 2018) feed the value head a stack of planes, because material is precisely the baseline that a value net must beat. So the binding constraint is **representation, not capacity**. `engine/state_encoder.py` supplies the planes: HP for each slot, active against benched, status, stat boosts, weather, terrain, Trick Room, Tailwind, screens, hazards, and the types of the active Pokemon.

Species identity is deliberately excluded. One-hot species across eight slots is a very large input, which on the games available would be fitted almost entirely to noise. Species enters only through type. The encoder is versioned, so that a change is visible rather than silent.

Two methodological points decide whether any of these numbers mean anything. **Splits are by game, never by state** — turns within a game share an outcome, so splitting on states leaks the label across the boundary and flatters every arm equally. And **every state is emitted twice, with the sides swapped**, because antisymmetry in the two players is a property of the game.

No figure from any of these models is published in this edition. Each one is downstream of MEDICHAM. Each one must be re-run before it is quoted.

4.10 Why the exploitability is the headline metric

For the decision itself, read ADR-003.

There are two kinds of game. In the first kind, both players see all of the state. Chess and Go are of this kind, and a search over that state is correct. In the second kind, each player holds private information. Poker is of this kind. VGC is also of this kind: you do not see which four of the six the opponent brings, the items, the abilities, or the fourth move of a set.

In the second kind of game, one best move does not exist. The correct object is a mixed strategy. If you always make the same choice in the same situation, an opponent who watches you can find that choice and defeat it. That is why the poker research of 2007 to 2021 produced CFR, DeepStack, Libratus and ReBeL, and not a larger chess search.

A fixed policy is therefore exploitable by construction. A behaviour clone is a fixed policy. A PPO agent is a fixed policy. VGC-Bench measured this on its own agents: it trained a best response against each agent and found approximately 100% exploitability, although one of those agents defeats a professional player. The two facts are consistent. Strength on average and readability under study are different quantities.

A win rate cannot show this defect. A win rate is measured against a population that does not adapt. An exploitability is measured against an opponent that is trained against you.

The claim under test. A search that computes a new answer each turn shows no fixed map to an opponent. It should therefore be harder to exploit than a compiled policy. Three properties of VGC can defeat that claim, and all three are open: simultaneous moves, stochastic resolution, and a short horizon.

Do not read this section as a result. The project has no exploitability figure. `data/exploitability.json` is declared void.

4.11 Support decisions. Do not predict outcomes.

In this format the winner of a game is close to impossible to predict from the two team sheets. ABRA therefore judges each model on a DECISION, not on the match result. Each probability ships a proper score, a confidence interval, and an honest baseline.

The confidence interval is clustered. States within one game are correlated, so an interval that resamples states would be too narrow. ABRA resamples whole games instead.

Honest negatives are kept. A negative that is measured is more useful than a positive that is asserted.

4.12 "Known failure" is a banned phrase

A red test is never a status. It is fixed in the session that saw it red, or it is waived by the owner by name. There is no third state.

A waiver lives in `data/test-waivers.json` with the owner's words and the date. `tests/run-all.js` prints it on every run as `WAIVED`. **A waived red is visible state. It is never a silent skip.**

This rule exists because a guard on the documentation rule was red on every run for two days and was reported each time as "one of the two known failures". Naming it *known* is what made it acceptable. **Say that the test is red and what you are doing about it, or fix it. Never file it.**

5. Appendix — procedures for a frozen engine

5.1 Description

An engine release is a copy of every file whose content can change a measured number. A measurement reads the copy. Other work on the repository does not change the copy.

The frozen set is declared as `SOURCES` in `engine/engine_release.js`. **Read it from there.**

```
node -e "console.log(require('./engine/engine_release.js').SOURCES.join('\n'))"
```

Do not copy that list into this page. It has grown four times: loader dependencies were added so that `REL.require` can resolve, lazily-read data files were added so that a snapshot can actually play a game, and the fitted weights were added because a claim about a model is a claim about one specific vector. A release that is a valid digest set but not a loadable engine is the failure that each addition fixed. A typed list here cannot track them.

The release identifier is a digest of the file digests. If the files do not change, the identifier does not change. A second cut of the same files makes no second copy.

5.2 Procedure — cut a release

1. Make sure that the files you want to freeze are correct.
2. Run the command. Give a reason.

```
node engine/engine_release.js cut "why this release exists"
```

1. Read the digests in the output. There is one for each entry in `SOURCES`. The count is whatever that declaration currently holds.
2. Confirm that the Showdown commit is not `UNKNOWN`. A cut refuses if the checkout is not at `PINNED_COMMIT`.

5.3 Procedure — measure against a release

1. Open the release at the start of the measurement.
2. Load every engine file from the release. Do not load it from `engine/`.
3. Put the stamp in the artifact that you write.

```
const REL = require('./engine_release.js').open();
const MEDI = REL.require('engine/medicham2-browser.js');
const artifact = Object.assign({}, REL.stamp(), result);
```

`REL.stamp()` writes `engine_release`, `showdown_commit` and `source_digests` into the artifact. `engine/provenance.js` reads `source_digests` and compares the `CONTENT`. An artifact without this stamp can be checked only by timestamp, and a timestamp cannot show that a file changed after it was read.

5.4 Procedure — pin the other two inputs

```
node engine/game_differential.js --release <id> --census <pin file> \
  --team-store data/team-pool-frozen --arm middle --steering empirical \
  --end-state --games 1200 --turns 50
```

Then prove that two samples are the same with `node engine/arms_comparable.js <before.json> <after.json> .`

5.5 Procedure — measure the three lattices

```
node engine/game_differential.js --steering empirical --release <id> --arm middle --end-state \
  --census data/mechanics-census.json --team-store data/team-pool-frozen \
  --games 1200 --write --out data/game-differential.json
# then --games 1350 --out data/game-differential.g1350.json
# then --games 1950 --out data/game-differential.g1950.json
```

Then run `SHOWDOWN_PATH=... node engine/quarantine.js .`

- **Do not read an artifact while another process writes it.** Read the modification time first.
- **Write the prediction before the run.** Compare after it.

5.6 Procedure — check a release

```
node engine/engine_release.js list          # each release, and how many files have moved since
node engine/engine_release.js verify <id>   # has the copy itself changed
node engine/engine_release.js compat <file> [symbol ...]
```

`list` shows the drift. Drift is normal. Drift tells you whether an old number still describes the engine that ships today.

5.7 Warning

Do not read `engine/` files during a measurement. Another division can write them at any time. This does not cause an error. It causes a number that is wrong and looks correct.

On 2026-08-04 three divisions ran at the same time with separate files. The weights of the model under test changed between the two halves of one measurement. The measurement completed. No check failed. 7,100 games were lost.

Do not delete a file that you did not create. An untracked file cannot be recovered. If something looks like debris, report it and leave it.

Kill only what you started, and kill it by process id. Never kill by image name. On this machine that ends every node process, including other divisions' work.

Companion documents. [White paper](#) · [Deck](#) · [Project summary](#) · [Model ledger](#) · [Running notes](#) · [Changelog](#)